📄 Project Documentation

✅ Title:

Development and Evaluation of a Machine Learning-Based Intrusion Detection System using the NSL-KDD Dataset

---

📑 Outline

---

1. Introduction

Intrusion Detection Systems (IDS) are security mechanisms designed to monitor network traffic and detect unauthorized access or malicious activity. With the continuous increase in cyber threats and sophisticated attack patterns, traditional rule-based IDS are proving inadequate. Machine learning (ML), with its ability to learn patterns and adapt over time, presents a promising alternative.

This project leverages supervised machine learning techniques to design and evaluate an IDS using a well-known dataset – NSL-KDD – aiming to distinguish between normal and malicious activity effectively.

---

2. Aim of the Project

The primary aim of this project is to develop a machine learning-based Intrusion Detection System (IDS) that can effectively detect malicious network activity with a low false positive rate.

---

3. Objectives

- **Literature Review**: Understand the current research and approaches to ML-based IDS.
- **System Development**: Build a functional IDS pipeline using the NSL-KDD dataset.
- **Model Implementation**: Train and evaluate selected ML models including a Decision Tree.
- **Performance Evaluation**: Assess each model using key metrics such as Accuracy, Precision, Recall, F1-Score, and False Positive Rate.

---

4. Technology Stack Used

- **Programming Language**: Python
- **Environment**: Jupyter Notebook
- **Virtual Environment**: `venv` (Python Virtual Environment)
- **Libraries**:
  - `pandas`, `numpy` – for data manipulation
  - `scikit-learn` – for model training and evaluation
  - `seaborn`, `matplotlib` – for data visualization
  - `tensorflow/keras` – for neural network implementation

---

5. Dataset Overview

The **NSL-KDD** dataset is a refined version of the KDD'99 dataset and is widely used in IDS research. It consists of labeled records that simulate different types of cyber-attacks along with normal traffic.

Each record includes numerical and categorical features describing aspects of the connection such as duration, protocol type, service, flag, and other network-level attributes. The final column indicates whether the connection is normal or an attack.

---

6. Data Preparation and Challenges

At the initial stage of the project, the training and testing data were kept separate based on the original structure of the NSL-KDD dataset. However, this led to a significant **performance drop**, especially in terms of precision and recall. Upon further analysis, it was discovered that the test set contained **attack patterns and categories that were absent in the training set**, which is not ideal for building generalizable models.

To address this:

- The **train and test datasets were merged**.
- The merged data was **randomized** to ensure balanced representation.
- A new **80-20 split** was done for training and testing.

This adjustment significantly improved the model's ability to generalize and detect attacks more reliably.

Additionally, categorical features such as protocol type, service, and flag were **one-hot encoded** to convert them into numerical form. Continuous features were **standardized** to ensure uniformity in scale – especially important for algorithm like Logistic Regression.

---

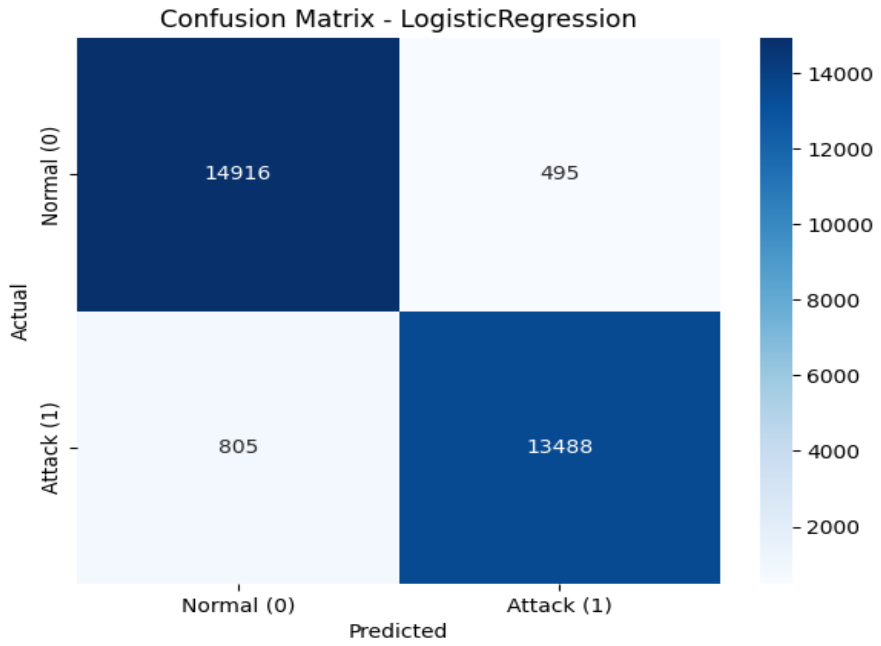
7. Machine Learning Models Implemented

◆ 1. Logistic Regression

A linear model used as a baseline. It tries to separate normal and malicious traffic using a linear boundary.

📄 Classification Report:

	precision	recall	f1-score	support
0	0.9488	0.9679	0.9582	15411
1	0.9646	0.9437	0.9540	14293
accuracy			0.9562	29704
macro avg	0.9567	0.9558	0.9561	29704
weighted avg	0.9564	0.9562	0.9562	29704

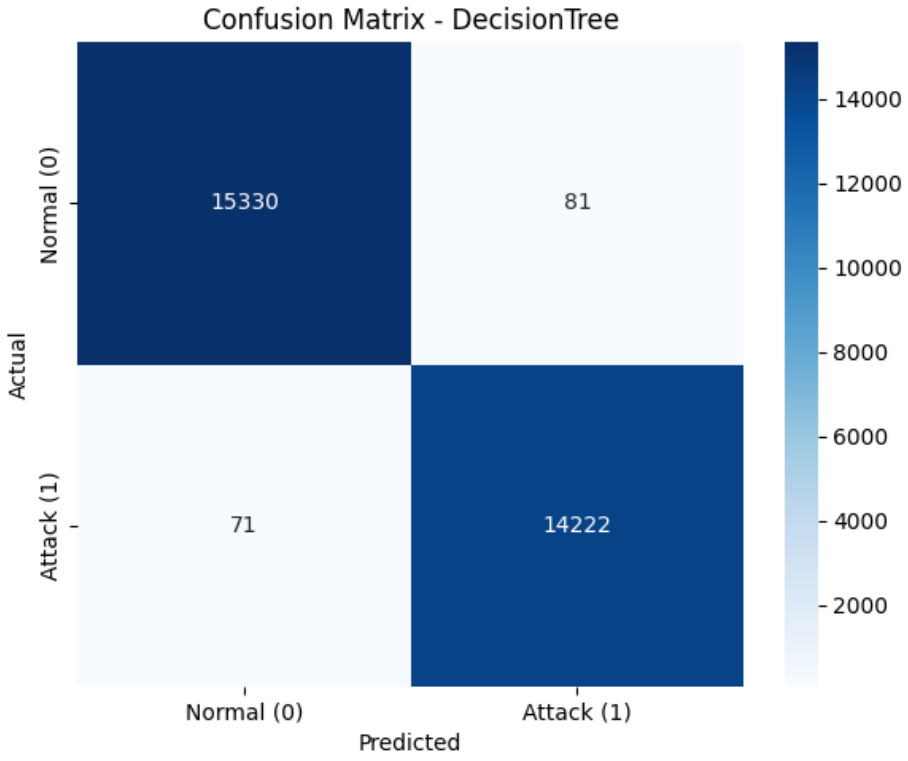


◆ 2. Decision Tree

A tree-structured model that splits data based on feature thresholds. It offers interpretability and performs well on structured data like NSL-KDD.

📄 Classification Report:

	precision	recall	f1-score	support
0	0.9954	0.9947	0.9951	15411
1	0.9943	0.9950	0.9947	14293
accuracy			0.9949	29704
macro avg	0.9949	0.9949	0.9949	29704
weighted avg	0.9949	0.9949	0.9949	29704

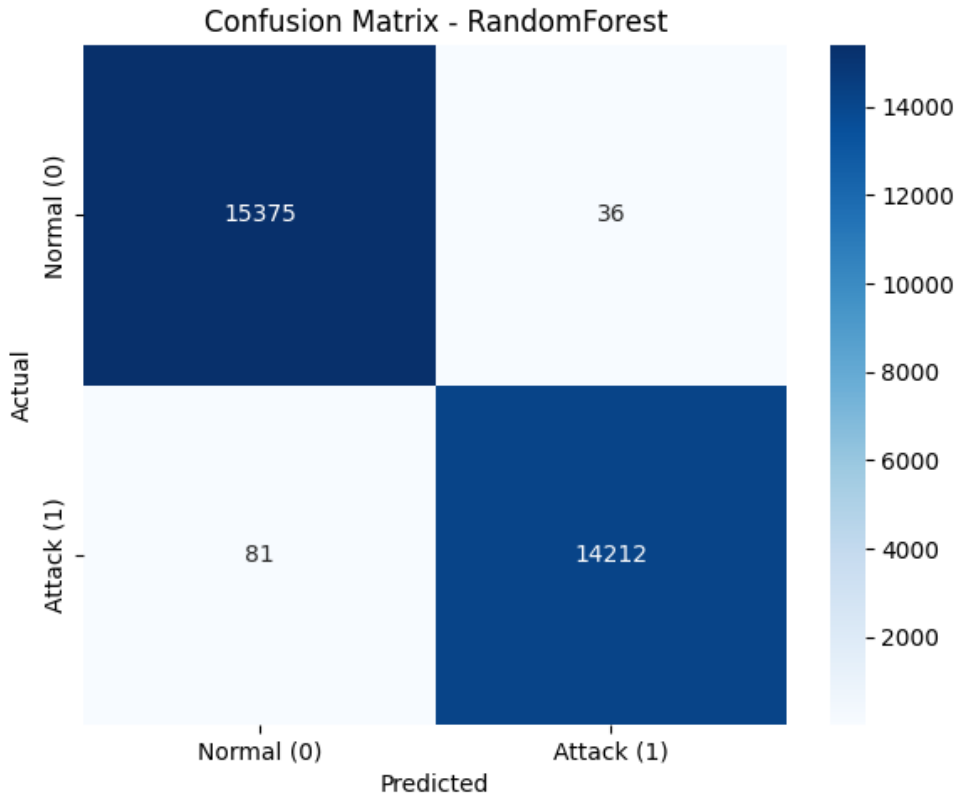


◆ 3. Random Forest

An ensemble of decision trees that reduces overfitting and increases accuracy. It demonstrated one of the best performances in this study.

Classification Report:

	precision	recall	f1-score	support
0	0.9948	0.9977	0.9962	15411
1	0.9975	0.9943	0.9959	14293
accuracy			0.9961	29704
macro avg	0.9961	0.9960	0.9961	29704
weighted avg	0.9961	0.9961	0.9961	29704



8. Performance Evaluation Metrics

Each model was assessed using the following metrics:

- **Accuracy**: Measures the overall correctness of predictions.
- **Precision**: How many predicted attacks were actually attacks.
- **Recall**: How many actual attacks were correctly predicted.
- **F1-Score**: A balance between precision and recall.
- **False Positive Rate (FPR)**: The rate at which normal traffic was wrongly classified as an attack.

These metrics are crucial because in real-world IDS applications, a **high false positive rate** can lead to alert fatigue and resource wastage, while **low recall** may mean actual threats go undetected.

---

9. Model-by-Model Performance Summary

Model	Accuracy	Precision	Recall	F1-Score	False Positive Rate
Logistic Regression	95.62%	96.46%	94.37%	95.40%	3.21%
Decision Tree	99.49%	99.43%	99.50%	99.47%	0.53%
Random Forest	99.61%	99.75%	99.43%	99.59%	0.23%

All models showed strong performance, but the **Random Forest** achieved the best balance of precision, recall, and minimal false positives.

---

10. Conclusion and Recommendation

The project successfully developed and evaluated a machine learning-based Intrusion Detection System using the NSL-KDD dataset. Several models were implemented and compared using meaningful metrics.

Key Findings:

- Merging and randomizing the dataset before splitting improved performance significantly.
- Random Forest proved to be the most effective model, showing high accuracy and extremely low false positive rates.
- Logistic Regression, while simple and interpretable, lagged behind in recall and false positive control.

Recommendation:
For real-time deployment of an IDS where minimizing false alarms is crucial, the **Random Forest** model is recommended due to its strong balance of all metrics and its robustness against unseen patterns.